Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en Ciencias y Sistemas

Título del Proyecto:

Park-Guard 2.0

PONDERACIÓN: 25 pts

🕐Tiempo estimado: 30 hrs

Índice

Índice **2**
1. Marco Formativo **3**
1.1. Valor 3
1.2. Competencia(s) 3
1.3. Objetivo SMART 4
2. Resumen Ejecutivo **5**
3. Enunciado del Proyecto **5**
3.1 Descripción del problema a resolver 5
3.2 Alcance del proyecto 6
Alcance obligatorio: 6
Alcance opcional: 7
Requerimientos técnicos: 8
3.3 Entregables 8
4. Material de apoyo **9**
Documentación técnica y plataformas oficiales 9
5. Metodología **9**
1. Investigación preliminar 9
2. Diseño del sistema 9
3. Montaje y construcción de la maqueta 10
4. Integración y programación 10
5. Validación y documentación final 10
6. Recursos y herramientas a utilizar **11**
7. Desarrollo de Habilidades Blandas **12**
8. Cronograma **13**
9. Rúbrica de Calificación **13**
9.1 Requisitos para optar a la calificación 13
9.2 Resumen de Puntuaciones 14
Detalle de la Calificación 15

1. Marco Formativo

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Se promueve el cumplimiento ético y técnico en el desarrollo del sistema, garantizando la correcta conexión, configuración y funcionamiento de cada componente electrónico.
Trabajo en equipo	Se impulsa la colaboración entre los integrantes del grupo para integrar las distintas partes del sistema, respetando las funciones asignadas y favoreciendo la comunicación efectiva.
Innovación	Se estimula la aplicación creativa de los conocimientos adquiridos en la implementación de soluciones automatizadas que mejoren la eficiencia de los procesos industriales.

1.2. Competencia(s)

Tipo de Competencia	Competencia
Competencia General	• Aplica estándares de calidad, eficiencia y seguridad en la implementación adecuada de soluciones de software, hardware y TIC en general. • Aplica principios básicos de ingeniería, ciencias de la computación y sistemas de información y comunicación en la formulación y resolución adecuada de problemas complejos. • Construye soluciones integrales trabajando en forma colaborativa y propositiva en equipos interdisciplinarios, de manera presencial o utilizando plataformas virtuales.
Competencia Específica	• Demuestra pensamiento crítico, actitud investigativa y rigor analítico en el planteamiento y la resolución de problemas complejos. • Aplica los conocimientos de su disciplina en la elaboración, fundamentación y defensa de argumentos para prevenir y resolver problemas complejos en su campo profesional, identificando y aplicando innovaciones. • Demuestra destreza y habilidad en la selección, uso y adaptación de herramientas metodológicas, tecnológicas y equipos especializados, así como en la lectura e interpretación de datos pertinentes al contexto de su ejercicio profesional.

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Desarrollar e implementar un sistema IoT distribuido denominado **Park-Guard 2.0**, capaz de gestionar el control de acceso vehicular, monitorear la ocupación de espacios en tiempo real y registrar eventos operativos mediante una arquitectura basada en MQTT, procesamiento en Python, base de datos NoSQL y dashboard web interactivo, permitiendo análisis histórico y supervisión remota del parqueo.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	El éxito del proyecto se verificará mediante el funcionamiento integral de la arquitectura IoT. Se evaluará con métricas como latencia de actualización, precisión en la detección de ocupación, consistencia de los registros almacenados y correcta ejecución del flujo completo sensor > broker > backend > base de datos > API > frontend.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	El proyecto se desarrollará utilizando Arduino o Raspberry Pi como nodo físico, un broker MQTT desplegado en contenedores Docker, un consumer en Python para procesamiento de eventos, una API con FastAPI o Flask y una base de datos NoSQL para almacenamiento flexible de datos. Las fases serán implementadas progresivamente en laboratorio, integrando hardware y software de forma modular y escalable.
Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	El sistema busca demostrar la integración de arquitecturas IoT modernas aplicadas a entornos urbanos inteligentes, permitiendo no solo la automatización local del parqueo, sino también la supervisión remota, análisis de patrones de ocupación, generación de eventos críticos y toma de decisiones basada en datos históricos, simulando un sistema real de gestión inteligente de infraestructura.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	El proyecto deberá completarse en un período de tres semanas, abarcando las fases de diseño de arquitectura IoT, implementación de comunicación MQTT, desarrollo del backend y base de datos NoSQL, creación del dashboard web, integración total del sistema y pruebas finales. Al concluir el período, el sistema deberá operar en tiempo real, almacenar datos históricos y presentar visualización analítica funcional.

2. Resumen Ejecutivo

El proyecto **Park-Guard 2.0: Sistema IoT Inteligente de Gestión y Analítica de Parqueos** se desarrolla dentro del curso de Arquitectura de Computadoras y Ensambladores 2 como una evolución del sistema automatizado local hacia una arquitectura distribuida basada en tecnologías IoT.

La problemática abordada no solo se limita a la gestión ineficiente de espacios de parqueo, sino también a la ausencia de sistemas integrados que permitan monitoreo remoto, análisis histórico de ocupación, detección automática de eventos críticos y generación de información estratégica para la toma de decisiones.

La propuesta consiste en diseñar e implementar un sistema físico de control de acceso y monitoreo de espacios que transmita datos en tiempo real mediante el protocolo MQTT hacia una arquitectura backend compuesta por un consumer en Python, una base de datos NoSQL y una API REST. Estos datos serán visualizados en un dashboard web interactivo que permitirá supervisión en tiempo real, consulta histórica y análisis de patrones de uso.

El sistema integrará sensores de ocupación, RFID para autenticación, sensores ambientales de seguridad y actuadores como talanqueras y alarmas. Además, registrará eventos operativos tales como accesos autorizados, intentos denegados, activaciones de emergencia y estadísticas de ocupación.

Con esta implementación, el proyecto trasciende la automatización local para convertirse en un sistema IoT escalable, orientado a eventos y preparado para simulaciones de entornos de ciudad inteligente.

3. Enunciado del Proyecto

3.1 Descripción del problema a resolver

En Guatemala, la gestión de parqueos en centros educativos, comerciales y empresariales suele realizarse de forma manual o con sistemas básicos que no ofrecen control en tiempo real sobre la disponibilidad de espacios, el acceso de usuarios ni las condiciones de seguridad del área. Esta situación provoca congestionamientos, pérdida de tiempo, errores humanos en el control de ingreso y salida, así como una baja capacidad de respuesta ante situaciones de emergencia como incendios, acumulación de gases o fallas operativas.

La ausencia de arquitecturas conectadas limita la capacidad de supervisión centralizada, impide detectar patrones de alta demanda, dificulta la generación de reportes operativos y reduce la capacidad de respuesta ante emergencias como acumulación de gases, incendios o fallos en los sistemas de acceso.

Desde una perspectiva tecnológica, muchos sistemas actuales funcionan como soluciones cerradas, sin comunicación distribuida, sin procesamiento de eventos y sin almacenamiento estructurado de información. Esto impide la implementación de modelos predictivos, auditorías digitales y escalabilidad hacia múltiples zonas o sedes.

Ante esta problemática, surge la necesidad de evolucionar el sistema Park-Guard hacia una arquitectura IoT moderna, donde los sensores y actuadores físicos no solo ejecuten acciones locales, sino que también publiquen eventos hacia una infraestructura de mensajería MQTT. Esta arquitectura permitirá el procesamiento en backend, almacenamiento en bases de datos NoSQL y visualización mediante dashboards web interactivos.

El nuevo enfoque transforma el parqueo en un sistema inteligente conectado, capaz de:

- Supervisar ocupación en tiempo real, de forma local.
- Registrar y consultar eventos históricos.
- Detectar condiciones de riesgo y generar alertas automáticas.
- Analizar patrones de uso para optimizar la gestión del espacio.
- Registrar nuevos usuarios en base a un identificador.
- Simular un modelo escalable aplicable a entornos de ciudad inteligente.

De esta forma, el proyecto no solo fortalece los principios de automatización y control embebido, sino que integra conceptos avanzados de sistemas distribuidos, arquitecturas orientadas a eventos y análisis de datos aplicados a infraestructura urbana.

3.2 Alcance del proyecto

El proyecto consiste en la evolución del sistema físico automatizado hacia una **arquitectura IoT distribuida basada en contenedores**, donde todos los servicios del sistema estarán desacoplados, dockerizados y comunicados mediante el protocolo MQTT.

El sistema integrará:

- Nodo físico (Arduino + ESP32 o Raspberry Pi)
- Broker MQTT
- Consumer en Python
- Backend API
- Base de datos NoSQL
- Frontend web
- Sistema de autenticación con JWT
- Generación de reportes
- Manejo dinámico de usuarios RFID

La arquitectura deberá permitir que el sistema sea fácilmente escalable y preparado para una futura migración a la nube.

El objetivo ya no se limita únicamente al funcionamiento físico de la maqueta, sino a la integración completa entre hardware y software mediante una infraestructura moderna, comunicación en tiempo real con MQTT, almacenamiento en base de datos NoSQL y visualización web segura con autenticación JWT.

El sistema deberá componerse de varios servicios desacoplados, ejecutándose en contenedores Docker independientes. La comunicación entre el nodo físico (Arduino + ESP32 o Raspberry Pi) y el backend se realizará exclusivamente mediante el protocolo MQTT, utilizando un broker desplegado en Docker. A su vez, los datos recibidos serán procesados por un consumer en Python, almacenados en una base de datos NoSQL también contenerizada y expuestos a través de una API REST segura.

El backend y el consumer deberán construirse como imágenes Docker personalizadas, publicarse en Docker Hub y utilizarse directamente desde dicha plataforma el día de la presentación. Este requisito tiene como finalidad garantizar que el proyecto esté preparado para su despliegue en la nube en la siguiente fase del curso. Durante la evaluación se solicitará una demostración en vivo del levantamiento del sistema utilizando docker-compose, evidenciando el dominio técnico del entorno de contenedores.

Desde el punto de vista funcional, el sistema deberá conservar la maqueta física desarrollada previamente, incluyendo los espacios de parqueo, las talanqueras automáticas, sensores de ocupación, sensor de gas o humo, ventilador, buzzer y pantalla LCD. Sin embargo, la lógica de validación de tarjetas RFID cambia: las tarjetas autorizadas ya no podrán estar definidas directamente en el código de Arduino o Raspberry. En esta fase, los usuarios y sus tarjetas deberán almacenarse en la base de datos y gestionarse dinámicamente desde el frontend.

Esto implica que el sistema deberá permitir registrar nuevas tarjetas, editar información de usuarios, activar o desactivar accesos y visualizar el listado completo de tarjetas registradas. Toda validación de acceso deberá consultar la base de datos a través del backend, reforzando el enfoque distribuido del proyecto.

Adicionalmente, el sistema deberá registrar y almacenar eventos operativos tales como ingresos autorizados, intentos denegados, salidas, ocupación y desocupación de espacios, activaciones de emergencia, encendido de ventilador y cambios de estado relevantes. Estos registros permitirán la generación de reportes descargables en formato PDF, incluyendo estadísticas básicas de uso y actividad del parqueo.

El frontend deberá desarrollarse utilizando React, Vue o Angular, incorporando autenticación mediante JWT para garantizar un acceso seguro. Todas las rutas protegidas deberán requerir token válido, y el backend deberá validar correctamente cada solicitud autenticada.

La visualización en tiempo real será un componente fundamental de esta fase. Utilizando MQTT con WebSockets en el frontend y la librería P5.js, se deberá representar gráficamente el estado de los espacios de parqueo, el nivel de gas ambiental, el estado de las talanqueras y los eventos recientes del sistema. Esta vista deberá actualizarse en tiempo real conforme se reciban mensajes del broker.

Desde el dashboard también se deberá permitir la ejecución de acciones remotas, como activar el ventilador o activar el sistema de emergencia, publicando los mensajes correspondientes al tópico MQTT adecuado. De esta forma, el sistema demuestra comunicación bidireccional completa entre interfaz web y dispositivo físico.

El proyecto también deberá incluir un mecanismo básico de detección de actividad sospechosa asociada a tarjetas RFID, por ejemplo, múltiples intentos de ingreso en un rango corto de tiempo. Estos eventos deberán registrarse en la base de datos y visualizarse como alertas en el sistema.

Asimismo, se deberá permitir la gestión administrativa de los espacios de parqueo, habilitando o deshabilitando espacios por mantenimiento u otras causas, reflejando estos cambios tanto en la base de datos como en la visualización en tiempo real.

En conjunto, esta fase busca que el estudiante integre conocimientos de automatización, sistemas distribuidos, bases de datos NoSQL, seguridad mediante autenticación con tokens, comunicación orientada a eventos y despliegue contenerizado. El resultado esperado es un sistema IoT funcional, modular, escalable y preparado para ser migrado a infraestructura en la nube en la siguiente etapa del proyecto.

Alcance opcional:

Podrá incorporarse funcionalidades adicionales para hacer del sistema un prototipo completo:

- Incorporación de LEDs por espacio, indicando libre u ocupado
- Contador de ingresos y salidas
- Posible expansión futura hacía comunicación inalámbrica o monitoreo remoto.
- Utilización de una interfaz gráfica utilizando processing.

Requerimientos técnicos:

El proyecto deberá implementarse como un sistema IoT distribuido, desacoplado y contenerizado, integrando hardware físico y servicios backend bajo una arquitectura orientada a eventos.

El sistema deberá garantizar coherencia entre las acciones físicas de la maqueta y su representación digital en tiempo real, manteniendo sincronización constante mediante el protocolo MQTT.

El sistema deberá ejecutarse bajo una arquitectura basada en contenedores Docker. Deberán ejecutarse en contenedores independientes:

- Broker MQTT
- Base de datos NoSQL
- Backend API
- Consumer MQTT
- Frontend

El backend, consumer y frontend deberán:

- Contener su propio Dockerfile.
- Construir una imagen personalizada.
- Publicarse en Docker Hub.
- Descargarse desde Docker Hub el día de la presentación.
- Integrarse mediante docker-compose.

Se evaluará el correcto uso de redes internas, variables de entorno y orquestación básica.

Durante la presentación se realizará una demostración en vivo del levantamiento del sistema completo utilizando Docker.

Comunicación en tiempo real

Toda la comunicación entre el nodo físico y el sistema deberá realizarse exclusivamente mediante MQTT.

No se permitirá comunicación directa HTTP entre el dispositivo físico y el backend.

El frontend deberá suscribirse a tópicos MQTT para visualización en tiempo real y publicar comandos cuando se activen acciones remotas como:

- Activación de ventilador.
- Activación de emergencia.
- Habilitar o deshabilitar parqueos

Documentación Obligatoria de Topics MQTT

El documento técnico deberá incluir una sección formal donde se describan:

- Lista completa de topics utilizados.
- Estructura jerárquica de los mismos.
- Formato JSON de cada mensaje.
- Descripción funcional de cada topic.
- Qué componente publica.
- Qué componente se suscribe.

Se evaluará la coherencia y organización del diseño de mensajería.

Diseño de Base de Datos NoSQL

Como mínimo deberán existir colecciones para:

- Usuarios RFID.
- Eventos.
- Espacios de parqueo.
- Logs del sistema.
- Estadísticas generadas.

Las tarjetas RFID no podrán estar almacenadas en el microcontrolador. La validación deberá realizarse consultando la base de datos a través del backend.

Seguridad - Autenticación JWT

El sistema deberá implementar autenticación mediante JSON Web Tokens (JWT).

Se requiere:

- Login obligatorio.
- Manejo seguro de credenciales.

Diseño de la Interfaz Web

El frontend deberá desarrollarse utilizando React, Vue o Angular. La interfaz deberá estar organizada en dashboards diferenciados y claramente estructurados:

Dashboard de Visualización en tiempo real

Debe incluir:

- Representación gráfica de los espacios ocupados utilizando P5.js.
- Nivel de gas ambiental.
- Estado del ventilador.
- Estado de emergencia.
- Estado de talanqueras.

- Espacios disponibles.
- Registro en vivo de eventos recientes.

La actualización deberá realizarse mediante suscripción MQTT.

Dashboard de Gestión de Usuarios

Debe permitir:

- Crear usuario con tarjeta RFID.
- Editar usuario.
- Activar o desactivar tarjeta.
- Visualizar historial básico de actividad.
- Identificar actividad sospechosa (múltiples intentos en corto tiempo).
- Manejo de saldo por usuario.

Dashboard de Estadísticas y Reportes

- Gráficas de ingresos y salidas.
- Espacio más utilizado.
- Cantidad de emergencias.
- Activaciones de ventilador.

Desde este dashboard deberá poder generarse un reporte en PDF descargable con las estadísticas mostradas.

3.3 Entregables

Tipo	Descripción
Manual de Usuario	Documento que describa el funcionamiento del sistema web, detallando funcionalidades y uso.
Documento técnico.	Agregar el diagrama que represente claramente la arquitectura del sistema. Nodo físico > Broker MQTT > Consumer > Base de Datos NoSQL > Backend API > Frontend. Incluir objetivos, alcance, arquitectura, documentación de topics, diseño de base de datos (esquemas) diseño de dashboards, explicación del flujo completo.
Maqueta Funcional	Sistema físico completamente operativo y sincronizado con el sistema IoT
Repositorio Git	Mínimo 2 commits por persona, en fechas diferentes.

Interfaz	Dashboards descritos funcionales y con interfaz intuitiva.

Penalizaciones
70% de penalización si no se cumplen las funcionalidades físicas de la Fase 1.
10% de penalización por no cumplir el mínimo de commits.
Penalización si las imágenes no están publicadas en Docker Hub.

4. Material de apoyo

Para facilitar el desarrollo del proyecto **Park-Guard 2.0: Sistema IoT Distribuido y Escalable**, se recomienda a los estudiantes consultar recursos técnicos orientados a arquitecturas distribuidas, comunicación basada en eventos, contenerización y desarrollo de aplicaciones web modernas.

En esta fase, el enfoque ya no se limita únicamente al control embebido con Arduino, sino que integra conceptos de sistemas IoT, bases de datos NoSQL, seguridad mediante autenticación con tokens, procesamiento backend y despliegue en contenedores.

Los siguientes materiales permitirán comprender tanto la parte física como la arquitectura completa del sistema.

Documentación oficial de Arduino — Referencia del lenguaje, manejo de interrupciones y librerías para sensores.
Raspberry Pi Documentation — Configuración de red, GPIO y conectividad WiFi.
Librerías MQTT para Arduino (PubSubClient u otras).
Documentación oficial de MQTT — Conceptos de publish/subscribe, QoS y estructura de tópicos.
Eclipse Mosquitto — Configuración y despliegue de broker MQTT.
Guías sobre diseño de jerarquía de topics y mensajes JSON.

5. Metodología

La metodología del proyecto **Park-Guard: Sistema automatizado de parqueo** se organiza en fases progresivas que guían al estudiante desde la investigación inicial hasta la validación del sistema automatizado. Este enfoque busca que los participantes comprendan no sólo qué deben construir, sino también cómo lograr un resultado funcional y bien documentado. Cada fase incluye actividades concretas que deberán registrarse mediante evidencias fotográficas y bitácoras de avance.

6. Recursos y herramientas a utilizar

Para el desarrollo del proyecto **Park-Guard: Sistema automatizado de parqueo**, los estudiantes deberán utilizar herramientas tecnológicas que faciliten la programación, el diseño, la simulación y la documentación del sistema. Los recursos seleccionados buscan reforzar la comprensión de los procesos de automatización y el trabajo colaborativo dentro de entornos académicos.

Software

- **Arduino IDE**: entorno principal para la programación del microcontrolador y la implementación de los subsistemas de control y seguridad.
- **Processing**: herramienta de apoyo para el diseño de la interfaz gráfica de visualización y la comunicación serial con el sistema físico.

- **TinkerCAD Circuits**: simulador en línea para pruebas iniciales de circuitos y componentes electrónicos.
- **Fritzing**: programa para elaborar diagramas de conexión eléctricos y esquemas de montaje.
- **Canva o PowerPoint**: para la elaboración de presentaciones visuales de avance y documentación final.
- **Microsoft Word o Google Docs**: para la redacción de la documentación técnica y la bitácora de desarrollo.

Plataformas de trabajo y gestión

- **UEDI**: plataforma institucional para la publicación de actividades, entregas y avisos del curso.
- **GitHub**: control de versiones y almacenamiento del código fuente, diagramas y documentación digital del proyecto.
- **Google Drive**: espacio para compartir archivos, fotografías y evidencias del trabajo en equipo.

7. Desarrollo de Habilidades Blandas

Además de los conocimientos técnicos relacionados con la automatización, el control y la integración de sistemas, el proyecto **Park-Guard** busca fortalecer competencias transversales esenciales para el ejercicio profesional. El trabajo en equipo, la comunicación efectiva y la capacidad de resolver problemas en grupo son elementos centrales del desarrollo del proyecto, ya que simulan entornos reales de trabajo en ingeniería.

Trabajo en equipo

El proyecto se desarrolla en grupos, asignando roles definidos a cada integrante. Entre los posibles roles se incluyen líder técnico, encargado de programación, responsable del diseño físico, coordinador de documentación y encargado de pruebas. Esta distribución permite que cada estudiante asuma una responsabilidad específica, promueva la colaboración y contribuya al cumplimiento de los objetivos del grupo.

Comunicación efectiva

Durante el desarrollo del proyecto, los equipos deberán realizar reportes de avance y presentaciones parciales que fomenten la claridad en la exposición de resultados y la retroalimentación constante. Se promoverá el uso de herramientas digitales para el seguimiento del progreso y la coordinación de tareas. La comunicación será tanto técnica como organizativa, reforzando la habilidad para explicar procesos complejos de manera comprensible.

Resolución de conflictos y toma de decisiones

El trabajo colaborativo puede implicar diferencias de criterio en el diseño o en la implementación. Los equipos deberán aplicar estrategias de negociación y consenso para resolver discrepancias, priorizando la objetividad y el cumplimiento del plan de trabajo. Se busca que los estudiantes desarrollen pensamiento crítico, adaptabilidad y una actitud profesional frente a los retos del trabajo grupal.

Gestión y liderazgo

Cada grupo deberá planificar su tiempo, distribuir tareas y coordinar la integración de los módulos. La correcta gestión del proyecto implicará el uso de cronogramas, listas de tareas y reuniones de revisión. Este proceso fomenta la responsabilidad compartida, la autonomía de los integrantes y el desarrollo de habilidades de liderazgo técnico y organizativo.

8. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	sábado 3 de marzo del 2025	sábado 21 de marzo del 2026
Elaboración	sábado 3 de marzo del 2026	viernes 20 de marzo del 2026
Calificación	sábado 21 de marzo del 2026	sábado 21 de marzo del 2026

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	El proyecto debe implementarse utilizando Arduino y mostrar la información en pantalla. No se aceptarán implementaciones con plataformas distintas.	
Uso de herramientas requeridas	Se debe evidenciar el uso de los entornos Arduino IDE y Processing, así como el registro del trabajo en GitHub o Drive según se indique.	
Construcción de maqueta	La maqueta debe representar todas las secciones del sistema. Debe ser visualmente clara, organizada y funcional.	
Interfaz de visualización	La interfaz en pantalla LCD debe mostrar en tiempo real los estados solicitados del sistema, incluyendo las alertas y el estado de ocupación del parqueo.	
Evidencias de desarrollo	Deben presentarse fotografías del proceso de construcción, pruebas de funcionamiento y bitácora con los problemas encontrados y las soluciones aplicadas.	

9.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
Competencia / Habilidad	**Puntos Totales**	**Puntos Obtenidos**
Uso correcto de Docker y Docker Compose	10	
Imagenes en docker hub y uso de tags	3	
Documentación Técnica	5	
Manual de usuario claro y funcional	5	
Orden del repositorio y mínimo 2 commits en fechas distintas	2	
Trabajo y presentación final	5	
Subtotal Habilidades	**30**	
2. Conocimiento		
Tema / Módulo	**Puntos Totales**	**Puntos Obtenidos**
Broker MQTT dockerizado y funcional	4	
Base de datos NoSQL dockerizada	4	
Backend dockerizado con imagen propia en Docker Hub	4	
Consumer dockerizado con imagen propia en Docker Hub	4	
Comunicación correcta vía MQTT (publish/subscribe)	6	
Procesamiento de eventos en consumer	4	
Registro correcto de eventos en base de datos	5	
Implementación correcta de JWT	4	
Gestión dinámica de usuarios RFID	8	

(CRUD completo)		
Detección de actividad sospechosa	3	
Dashboard en tiempo real con P5.js	10	
Dashboard de estadísticas con gráficas	5	
Generación de reporte en PDF	4	
Activación remota (ventilador / emergencia, parqueos) vía MQTT	5	
Subtotal Conocimiento	**70**	
TOTAL	**100**	

Detalle de la Calificación

Criterio/Nivel **Nivel de evaluación**

No.	Criterio de evaluación	Punteo máximo		Satisfactorio 100% – 61%	Necesita mejorar 60% – 0%	Punteo obtenido
1	**Habilidades**					
1.1	Uso correcto de Docker y Docker Compose	10	Descripción	El sistema se levanta completamente con docker compose up. Todos los servicios funcionan en red interna. Uso correcto de variables de entorno, volúmenes y dependencias. No requiere configuraciones manuales adicionales. Prueba funcional en vivo.	Fallas en la orquestación, errores de conexión entre contenedores, configuración manual fuera de docker-compose o servicios que no inician correctamente.	
			Rango del punteo	10 – 6	5 – 0	

1.2	Publicación de imágenes en Docker Hub	3	Descripción	Backend, consumer y frontend publicados en Docker Hub con uso correcto de tags (v1.0, latest). Se descargan exitosamente el día de la presentación.	No se publicaron imágenes o no funcionan al descargarlas. Uso incorrecto de tags.	
			Rango del punteo	3 – 2	1 – 0	
1.3	Documentación técnica	5	Descripción	Incluye arquitectura, diagrama completo del sistema, documentación formal de topics MQTT, diseño de base de datos, explicación del flujo completo y justificación técnica.	Documento incompleto, sin diagrama claro o sin documentación detallada de la arquitectura.	
			Rango del punteo	5 – 3	2 – 0	
1.4	Manual de usuario	5	Descripción	Explica login, dashboards, gestión de usuarios RFID, generación de reportes y acciones remotas paso a paso con capturas o ejemplos claros.	Manual superficial, incompleto o difícil de entender.	

			Rango del punteo	5 – 3	2 – 0	
1.5	Orden del repositorio y mínimo 2 commits en fechas distintas	2	Descripción	Repositorio estructurado por servicios. Mínimo 2 commits por integrante en fechas distintas.	No cumple mínimo de commits o estructura desordenada.	
			Rango del punteo	2	1 – 0	
1.6	Trabajo y presentación final	5	Descripción	Explican arquitectura IoT, MQTT, JWT, Docker, flujo de eventos y demuestran funcionamiento en vivo sin fallos críticos.	Presentación confusa o sin dominio del sistema interno.	
			Rango del punteo	5 – 3	2 – 0	
	Sub-Total de Puntos	**30**				
2	**Conocimientos**					

2.1	Broker MQTT dockerizado y funcional	4	Descripción	Broker ejecutándose en contenedor independiente, permitiendo publish/subscribe entre nodo físico, consumer y frontend.	Broker mal configurado o dependiente de instalación externa fuera de Docker.	
			Rango del punteo	4 – 3	2 – 0	
2.2	Base de datos NoSQL dockerizada	4	Descripción	Base de datos ejecutándose en contenedor independiente con persistencia mediante volúmenes. Estructura coherente de colecciones (usuarios, eventos, espacios, logs, estadísticas).	Base de datos sin persistencia, mal estructurada o dependiente de instalación externa.	
			Rango del punteo	4 – 3	2 – 0	
2.3	Backend dockerizado con imagen propia en Docker Hub	4	Descripción	Backend construido con Dockerfile propio, publicado en Docker Hub y desplegado exitosamente (uso de logs) mediante docker-compose.	Imagen no publicada, errores en ejecución o dependencia de entorno local.	
			Rango del punteo	4 – 3	2 – 0	

2.4	Consumer dockerizado con imagen propia en Docker Hub	4	Descripción	Consumer suscrito a los tópicos definidos, procesa mensajes correctamente y está publicado en Docker Hub. (uso de logs).	No procesa eventos correctamente o no está contenerizado adecuadamente.	
			Rango del punteo	4 – 3	2 – 0	
2.5	Comunicación correcta vía MQTT (publish/subscribe)	6	Descripción	Toda la comunicación entre nodo físico, consumer y frontend se realiza exclusivamente vía MQTT. Topics jerárquicos bien definidos y mensajes en formato JSON estructurado.	Uso parcial de HTTP o estructura incorrecta de topics y mensajes.	
			Rango del punteo	6 – 4	3 – 0	
2.6	Procesamiento de eventos en consumers	4	Descripción	Clasifica eventos (accesos, emergencias, ocupación, intentos fallidos) y ejecuta acciones correspondientes antes de almacenar.	Procesamiento incompleto o eventos mal clasificados.	
			Rango del punteo	4 – 3	2 – 0	

2.7	Registro correcto de eventos en base de datos	5	Descripción	Todos los eventos quedan registrados con timestamp, tipo y referencia de usuario o espacio. Consultables desde el backend.	Registros incompletos o inconsistentes.	
			Rango del punteo	5 – 3	2 – 0	
2.8	Implementación correcta de JWT	4	Descripción	Login obligatorio, rutas protegidas, validación correcta del token en backend y manejo seguro de credenciales.	Rutas sin protección o token mal implementado.	
			Rango del punteo	4 – 3	2 – 0	
2.9	Gestión dinámica de usuarios RFID (CRUD completo)	8	Descripción	Permite crear, editar, activar/desactivar y eliminar usuarios desde el frontend. Validación de acceso consultando base de datos, no almacenada en microcontrolador.	CRUD incompleto o validación local en hardware.	
			Rango del punteo	8 – 5	4 – 0	
2.10	Detección de actividad sospechosa	3	Descripción	Detecta múltiples intentos fallidos en corto tiempo y genera alerta registrada en base de datos y dashboard.	No detecta intentos repetidos o no genera alerta.	
			Rango del punteo	3 – 2	1 – 0	

2.11	Dashboard en tiempo real con P5.js	10	Descripción	Representación gráfica dinámica mediante P5.js actualizada vía MQTT WebSockets. Refleja ocupación, estado de talanqueras, ventilador y emergencias en tiempo real.	Visualización parcial o sin actualización en tiempo real.	
			Rango del punteo	10 – 7	6 – 0	
2.12	Dashboard de estadísticas con gráficas	**5**	Descripción	Incluye gráficas de ingresos, ocupación, emergencias y uso del ventilador con datos reales almacenados.	Estadísticas incompletas o sin datos reales.	
			Rango del punteo	5 – 3	2 – 0	
2.13	Generación de reporte en PDF	4	Descripción	Genera reporte PDF descargable con resumen estadístico y datos del sistema.	PDF incompleto o sin información relevante.	
			Rango del punteo	4 – 3	2 – 0	
2.14	Activación remota (ventilador / emergencia, parqueos) vía MQTT	5	Descripción	Desde el frontend se publican comandos MQTT que activan ventilador, emergencia o habilitación/deshabilitación de espacios, reflejándose inmediatamente en el hardware.	Activación parcial o sin sincronización con el sistema físico.	

			Rango del punteo	5 – 3	2 – 0	
	Sub-Total de Puntos	**70**				
	Total General	**100**				